

DNA DB - Documentation Complète des Syntaxes



TABLE DES SYNTAXES

1. Syntaxe de Création
 2. Syntaxe des Types
 3. Syntaxe des Mutations
 4. Syntaxe des Requêtes
 5. Syntaxe de Configuration
 6. Syntaxe du Stockage
 7. Syntaxe des Index
 8. Syntaxe du Fitness
 9. Syntaxe des Signatures
 10. Syntaxe des Macros
-

1. SYNTAXE DE CRÉATION

1.1 Création d'une Base de Données

rust

// Syntaxe de base

```
let db = DNADatabase::builder()
    .data_dir("./dna_data")
    .build()?;
```

// Syntaxe complète

```
let db = DNADatabase::builder()
    .data_dir("./my_database")
    .cache_size(5000)
    .mmap_threshold(2 * 1024 * 1024)
    .auto_checkpoint_delta_count(50)
    .fitness_config(FitnessConfig::new())
    .build()?;
```

// Syntaxe avec configuration personnalisée

```
let config = DNADatabaseConfig {
    data_dir: "./data".to_string(),
    cache_size: 10000,
```

```

        mmap_threshold_bytes: 1024 * 1024,
        wal_max_file_size: 20 * 1024 * 1024,
        auto_checkpoint_delta_count: 100,
        fitness_config: FitnessConfig::default(),
    };
let db = DNADatabase::new(config)?;

```

1.2 Création de Valeurs

```

rust

// Syntaxe des valeurs numériques
let int_val = Value::Int(42);
let int_negatif = Value::Int(-100);
let int_max = Value::Int(9_223_372_036_854_775_807);

let float_val = Value::Float(3.14159);
let float_scientifique = Value::Float(1.5e-10);

// Syntaxe des chaînes
let string_val = Value::String("Hello World".to_string());
let string_vide = Value::String(String::new());
let string_multiligne = Value::String("Ligne 1\nLigne 2".to_string());

// Syntaxe des booléens
let bool_true = Value::Bool(true);
let bool_false = Value::Bool(false);

// Syntaxe des bytes
let bytes_val = Value::Bytes(vec![0x00, 0xFF, 0xDE, 0xAD]);
let bytes_vide = Value::Bytes(Vec::new());

// Syntaxe null
let null_val = Value::Null;

// Syntaxe de conversion
let from_int = Value::from(42); // Implémente From traits
let from_float = Value::from(3.14);

```

1.3 Création de Gènes

```

rust

// Syntaxe standard

```

```

let gene = Gene::new(
    1,
    "user_name".to_string(),
    Value::String("Alice".to_string()),
    0.8,
);

// Syntaxe avec poids par défaut (0.5)
let gene_default = Gene::with_default_weight(
    2,
    "user_age".to_string(),
    Value::Int(30),
);

// Syntaxe raccourcie (si From traits implémentés)
let gene_short = Gene {
    id: 3,
    name: "active".to_string(),
    value: Value::Bool(true),
    weight: 0.9,
};

```

1.4 Création de Chromosomes

```

rust

// Syntaxe standard
let chromo = Chromosome::new(
    1,
    "profile".to_string(),
    vec![1, 2, 3],
    0.01, // 1% de mutation
);

// Syntaxe vide
let chromo_vide = Chromosome::empty(2, "empty_section".to_string());

// Syntaxe avec ajout progressif
let mut chromo_builder = Chromosome::empty(3, "dynamic".to_string());
chromo_builder.add_gene(10);
chromo_builder.add_gene(11);
chromo_builder.add_gene(12);
chromo_builder.remove_gene(11); // Supprime le gène 11

```

```
// Syntaxe de vérification
if chromo.has_gene(1) {
    println!("Le gène 1 est présent");
}
```

1.5 Création de Genomes

rust

```
// Syntaxe de base
let genome = Genome::new(
    1,
    vec![chromosome1, chromosome2],
    vec![gene1, gene2, gene3],
);

// Syntaxe avec enfant (mutation)
let child = parent_genome.child(new_id);

// Syntaxe manuelle
let genome_manuel = Genome {
    id: 1,
    parent_id: None,
    generation: 1,
    chromosomes: vec![chromo],
    genes: HashMap::new(),
    fitness: FitnessScore::new(0.5),
    memory_status: MemoryStatus::Active,
    created_at: 1234567890,
    last_access: 1234567890,
    access_count: 0,
    delta_count_since_checkpoint: 0,
};
```

2. SYNTAXE DES TYPES

2.1 Énumération Value

rust

```
// Pattern matching sur Value
```

```

match gene.value {
    Value::Int(v) => println!("Entier: {}", v),
    Value::Float(v) => println!("Flottant: {}", v),
    Value::String(v) => println!("Chaîne: {}", v),
    Value::Bool(v) => println!("Booléen: {}", v),
    Value::Bytes(v) => println!("Bytes: {} octets", v.len()),
    Value::Null => println!("Null"),
}

// Extraction de valeurs
if let Value::Int(age) = gene.value {
    if age > 18 {
        println!("Majeur");
    }
}

// Vérification de type
if gene.value.type_id() == 0 {
    println!("C'est un entier");
}

```

2.2 Type FitnessScore

```

rust

// Syntaxe de création
let fitness = FitnessScore::new(0.75);
let fitness_auto = FitnessScore::new(0.95);
let fitness_min = FitnessScore::new(-1.0);    // → 0.0
let fitness_max = FitnessScore::new(2.0);    // → 1.0

// Syntaxe d'accès
let valeur = fitness.value();
println!("Score: {:.2}", valeur);

// Syntaxe de comparaison
if fitness1 > fitness2 {
    println!("Le premier est meilleur");
}

// Syntaxe d'addition (si implémenté)
let somme = fitness1.value() + fitness2.value();

```

2.3 Type MemoryStatus

```
rust

// Syntaxe de création
let status = MemoryStatus::from_fitness(0.8); // → Active
let status = MemoryStatus::from_fitness(0.5); // → Passive
let status = MemoryStatus::from_fitness(0.2); // → Fossil

// Syntaxe manuelle
let active = MemoryStatus::Active;
let passive = MemoryStatus::Passive;
let fossil = MemoryStatus::Fossil;

// Pattern matching
match genome.memory_status {
    MemoryStatus::Active => println!("En mémoire vive"),
    MemoryStatus::Passive => println!("Sur disque"),
    MemoryStatus::Fossil => println!("Archivé"),
}
```

3. SYNTAXE DES MUTATIONS

3.1 Syntaxe des Types de Mutations

```
rust

// Substitution de gène
let mutation_substitute = Mutation::SubstituteGene {
    genome_id: 1,
    chromosome_id: 1,
    gene_id: 5,
    new_value: Value::String("new_value".to_string()),
};

// Insertion de gène
let mutation_insert = Mutation::InsertGene {
    genome_id: 1,
    chromosome_id: 1,
    gene: Gene::new(100, "new_gene".to_string(), Value::Int(42), 0.5),
    position: Some(0), // Au début
    // position: None, // À la fin
}
```

```
};

// Suppression de gène
let mutation_delete = Mutation::DeleteGene {
    genome_id: 1,
    chromosome_id: 1,
    gene_id: 5,
};

// Insertion de chromosome
let mutation_insert_chromo = Mutation::InsertChromosome {
    genome_id: 1,
    chromosome: Chromosome::new(10, "new_chromo".to_string(), vec![], 0.05),
    position: None,
};

// Suppression de chromosome
let mutation_delete_chromo = Mutation::DeleteChromosome {
    genome_id: 1,
    chromosome_id: 10,
};

// No-op (aucune mutation)
let mutation_noop = Mutation::Noop;
```

3.2 Syntaxe d'Application

```
rust

// Application simple
let new_id = db.mutate(mutation)?;

// Application avec vérification
if let Ok(new_id) = db.mutate(mutation) {
    println!("Mutation réussie: new_id={}", new_id);
}

// Application en chaîne
let id1 = db.create_genome(...)?;
let id2 = db.mutate(Mutation::SubstituteGene { ... })?;
let id3 = db.mutate(Mutation::InsertGene { ... })?;
```

3.3 Syntaxe des Mutations Aléatoires

```
rust

// Générer une mutation aléatoire
let random_mut = mutation_engine.random_mutation(&genome);

// Appliquer une mutation aléatoire si probabilité
if rand::random::<f32>() < 0.1 {
    let mutation = mutation_engine.random_mutation(&genome);
    db.mutate(mutation)?;
}
```

3.4 Syntaxe des Deltas

```
rust

// Création de delta
let mut delta = DeltaBuilder::new(new_id, parent_id, generation);

// Ajout d'opérations
delta.substitute_gene(chromo_id, gene_id, old_val, new_val);
delta.insert_gene(chromo_id, gene, position);
delta.delete_gene(chromo_id, gene_id);
delta.insert_chromosome(chromosome, position);
delta.delete_chromosome(chromo_id);

// Construction finale
let delta = delta.build();

// Vérification
if delta.is_empty() {
    println!("Delta vide");
}
```

4. SYNTAXE DES REQUÊTES

4.1 Lecture Simple

```
rust

// Lecture par ID
```



```

let genome = db.get_genome(123)?;

// Lecture avec vérification
match db.get_genome(id)? {
  Some(genome) => {
    println!("Trouvé: {}", genome.id);
  }
  None => {
    println!("Non trouvé");
  }
}

// Lecture avec accès enregistré
if let Some(mut genome) = db.get_genome(id)? {
  genome.record_access();
  // Utilisation de genome...
}

```

4.2 Recherche par Fitness

```

rust

// Top N genomes
let top_10 = db.top_genomes(10);
let top_100 = db.top_genomes(100);

// Parcours des meilleurs
for id in db.top_genomes(5) {
  if let Some(genome) = db.get_genome(id)? {
    println!("ID: {}, Fitness: {:.2}", id, genome.fitness.value());
  }
}

```

4.3 Recherche par Similarité

```

rust

// Recherche basique
let similar = db.search_similar(target_id, 0.7)?;

// Recherche avec seuil
let haute_similarite = db.search_similar(id, 0.9)?;
let basse_similarite = db.search_similar(id, 0.3)?;

```

```
// Recherche et filtrage
let similar = db.search_similar(target_id, 0.6)?
    .into_iter()
    .filter(|&id| id != target_id)
    .collect::<Vec<_>>();
```

4.4 Parcours de Genomes

```
rust

// Parcours de tous Les IDs (créés)
let max_id = mutation_engine.next_id();
for id in 1..max_id {
    if let Some(genome) = db.get_genome(id)? {
        // Traitement...
    }
}

// Parcours des top genomes
for id in db.top_genomes(50) {
    process_genome(id);
}
```

5. SYNTAXE DE CONFIGURATION

5.1 Configuration du Fitness

```
rust

// Configuration par défaut
let config = FitnessConfig::default();

// Configuration personnalisée
let config = FitnessConfig::new()
    .with_weights(
        0.5, // access_frequency
        0.2, // recency
        0.2, // query_match
        0.1, // structural
    );
```

```
// Configuration manuelle
let config = FitnessConfig {
    weight_access_frequency: 0.4,
    weight_recency: 0.3,
    weight_query_match: 0.2,
    weight_structural: 0.1,
};
```

5.2 Configuration de la Base

```
rust

// Builder syntax
let db = DNADatabase::builder()
    .data_dir("./data")
    .cache_size(5000)
    .mmap_threshold(2_000_000)
    .auto_checkpoint_delta_count(50)
    .fitness_config(fitness_config)
    .build()?;

// Configuration struct
let config = DNADatabaseConfig {
    data_dir: "./data".to_string(),
    cache_size: 1000,
    mmap_threshold_bytes: 1_000_000,
    wal_max_file_size: 10_000_000,
    auto_checkpoint_delta_count: 90,
    fitness_config: FitnessConfig::default(),
};
```

6. SYNTAXE DU STOCKAGE

6.1 Opérations de Base

```
rust

// Sauvegarde
storage.save_genome(&genome)?;
storage.save_delta(&delta)?;
```

```

// Chargement
let genome = storage.load_genome(id)?;

// Vérification d'existence
let exists = storage.exists(id);

// Suppression
storage.delete_genome(id)?;

// Checkpoint manuel
storage.create_checkpoint(&genome)?;

```

6.2 Syntaxe Smart Storage

```

rust

// Le smart storage choisit automatiquement le backend
let smart = SmartStorage::new("./data", 1_000_000)?;

// Sauvegarde (backend automatique)
smart.save_genome(&genome)?;

// Chargement (backend automatique)
let genome = smart.load_genome(id)?;

// Delta
smart.save_delta(&delta)?;

```

6.3 Syntaxe WAL

```

rust

// Création d'entrée
let entry = WalEntry::new(&mutation, result_id, sequence);

// Append
wal.append(&entry)?;

// Lecture de toutes les entrées
let entries = wal.read_all()?;

// Parcours des entrées
for entry in wal.read_all()? {
    println!("Sequence: {}", entry.sequence);
}

```

```
println!("Timestamp: {}", entry.timestamp);
}
```

7. SYNTAXE DES INDEX

7.1 Index Global

```
rust

// Création
let index = GlobalIndex::new();

// Insertion
index.insert(genome);

// Recherche
let genome = index.get(id);

// Mise à jour
index.update(genome);

// Suppression
index.remove(id);

// Top fitness
let top = index.top_fitness(10);
```

7.2 Signatures Génétiques

```
rust

// Génération de signature
let sig = signature_gen.signature(&genome);

// Comparaison
let similarity = signature_gen.similarity(sig1, sig2);

// Seuils de similarité
if similarity > 0.8 {
    println("Très similaire");
} else if similarity > 0.5 {
```

```
        println("Moyennement similaire");
    } else {
        println("Différent");
    }
}
```

7.3 Recherche par Shard

```
rust

// Accès direct au shard
let shard = index.shard_for(id);

// Opération sur shard
shard.insert(genome, signature);
shard.update(genome, signature);
shard.remove(id);

// Top fitness par shard
let top = shard.top_fitness(10);

// Recherche par signature dans shard
let results = shard.search_by_signature(sig, 0.7, &sig_gen);
```

8. SYNTAXE DU FITNESS

8.1 Calcul Manuel

```
rust

// Calcul simple
let fitness = calculator.compute(&genome, None);

// Calcul avec score de requête
let fitness = calculator.compute(&genome, Some(0.9));

// Mise à jour in-place
calculator.update(&mut genome, None);
```

8.2 Mise à Jour dans DB

```
rust
```

```
// Mise à jour automatique (après accès)
db.get_genome(id)?; // Met à jour fitness

// Mise à jour manuelle
db.update_fitness(id, None)?;
db.update_fitness(id, Some(0.85))?;
```

8.3 Sélection Naturelle

```
rust

// Suppression des faibles
let removed = db.gc()?;

// Évolution
let mutations = db.evolve(10, 0.2)?;

// Statistiques
let stats = db.stats()?;
println!("Total: {}", stats.total_genomes);
println!("Actifs: {}", stats.active_count);
```

9. SYNTAXE DES SIGNATURES

9.1 Génération

```
rust

// Création du générateur
let generator = SignatureGenerator::new();

// Génération de signature
let sig = generator.signature(&genome);

// Affichage
println!("Signature: {:016b}", sig.0);

// Stockage
let sig_bytes = sig.0.to_le_bytes();
```

9.2 Comparaison

```
rust

// Similarité
let sim = generator.similarity(sig1, sig2);

// Égalité approximative
if generator.similarity(sig1, sig2) > 0.8 {
    println!("Genomes très similaires");
}

// Distance de Hamming
let hamming = (sig1.0 ^ sig2.0).count_ones();
println!("Distance: {} bits", hamming);
```

9.3 Recherche Brute force

```
rust

// Recherche dans une liste
let candidates = vec![genome1, genome2, genome3];
let similar = generator.find_similar_bruteforce(&target, &candidates, 0.7);
```

10. SYNTAXE DES MACROS

10.1 Macro de Création Rapide (à implémenter)

```
rust

// Proposition de macro (non encore implémentée)
// dna_create_gene!(id, name, value, weight)

// Usage proposé:
let gene = dna_create_gene!(1, "name", "Alice", 0.8);
let gene = dna_create_gene!(2, "age", 30, 0.6);
let gene = dna_create_gene!(3, "active", true, 0.5);
```

10.2 Macro de Requête (à implémenter)

```
rust
```



```
// Proposition de DNA-QL (à implémenter)
// dna_query!(db, SELECT * FROM genomes WHERE fitness > 0.7)

// Usage proposé:
// let results = dna_query!(db, {
//     SELECT name, age
//     FROM genomes
//     WHERE fitness > 0.5
//     ORDER BY fitness DESC
//     LIMIT 10
// });
```

10.3 Macro de Mutation (à implémenter)

```
rust

// Proposition (à implémenter)
// dna_mutate!(db, genome_id, SUBSTITUTE gene_id WITH value)

// Usage proposé:
// let new_id = dna_mutate!(db, 1, SUBSTITUTE 5 WITH "new_value");
// let new_id = dna_mutate!(db, 1, INSERT GENE (6, "new", 42, 0.5) INTO 1);
// let new_id = dna_mutate!(db, 1, DELETE GENE 5);
```

10.4 Série de Tests (existant)

```
rust

// Macro de test (dans les tests)
#[test]
fn test_example() {
    // Test code
}

// Parametric tests
#[test_case(1, 2, 3)]
#[test_case(4, 5, 9)]
fn test_addition(a: i32, b: i32, expected: i32) {
    assert_eq!(a + b, expected);
}
```

11. SYNTAXE DES OPÉRATEURS (Extensions Futures)

11.1 Opérateurs de Comparaison

```
rust

// Proposition (à implémenter avec des traits)
// if genome1 > genome2 { ... }
// if genome1.fitness > 0.7 { ... }

// Actuellement
if genome1.fitness.value() > genome2.fitness.value() { ... }
```

11.2 Opérateurs d'Addition (Fusion)

```
rust

// Proposition: Fusion de deux genomes
// let hybrid = genome1 + genome2;

// Actuellement (à implémenter)
// let hybrid = db.recombine(genome1, genome2)?;
```

11.3 Indexing Syntax

```
rust

// Proposition
// let gene = genome["name"];
// genome["age"] = Value::Int(31);

// Actuellement
let gene = genome.get_gene_by_name("name");
```

12. SYNTAXE DES FICHIERS DE CONFIGURATION

12.1 Format TOML (Proposition)

```
toml
```

```
# dna_config.toml
[database]
data_dir = "./dna_data"
cache_size = 5000
mmap_threshold_bytes = 1048576
auto_checkpoint_delta_count = 90

[fitness]
weight_access_frequency = 0.4
weight_recency = 0.3
weight_query_match = 0.2
weight_structural = 0.1

[storage]
wal_max_file_size = 10485760
compression = false

[performance]
threads = 4
shard_count = 256
```

12.2 Chargement de Configuration

```
rust

// Proposition
let config = DNADatabaseConfig::from_toml("config.toml");
let db = DNADatabase::new(config);
```

13. SYNTAXE DES COMMANDES CLI (Proposition)

```
bash

# Création d'une base
dna_db create --path ./mydb

# Import de données
dna_db import --db ./mydb --file data.json

# Requête
dna_db query --db ./mydb "SELECT * FROM genomes WHERE fitness > 0.7"

# Mutation
```

```
dna_db mutate --db ./mydb --genome 123 --substitute gene_5 "new_value"
```

```
# Statistiques
```

```
dna_db stats --db ./mydb
```

```
# Garbage collection
```

```
dna_db gc --db ./mydb
```

```
# Évolution
```

```
dna_db evolve --db ./mydb --generations 10 --prob 0.2
```

14. SYNTAXE DE SÉRIALISATION

14.1 Format Binaire (Interne)

```
rust
```

```
// Format d'entête (8 bytes)
// [0-3] Magic: "DNAD"
// [4]   Version: 0x01
// [5]   Type: 0x00 (checkpoint) / 0x01 (delta)
// [6-7] Reserved: 0x0000
```

```
// Format d'un gène
// [0-7]   id: u64
// [8-11]  name_len: u32
// [12-N]  name: bytes
// [N+1]   value_type: u8
// [N+2-N+9] value: dépend du type
// [N+10-N+13] weight: f32
```

14.2 Format JSON (Import/Export)

```
rust
```

```
// Export JSON
```

```
let json = serde_json::to_string(&genome)?;
```

```
// Import JSON
```

```
let genome: Genome = serde_json::from_str(json)?;
```

```
// Format JSON exemple
{
  "id": 1,
  "generation": 1,
  "fitness": 0.5,
  "chromosomes": [
    {
      "id": 1,
      "name": "profile",
      "genes": [1, 2, 3]
    }
  ],
  "genes": {
    "1": {
      "name": "name",
      "value": {"String": "Alice"},
      "weight": 0.8
    }
  }
}
```

15. SYNTAXE DES TESTS

15.1 Test Unitaire

```
rust

#[test]
fn test_gene_creation() {
    let gene = Gene::new(1, "test".to_string(), Value::Int(42), 0.5);
    assert_eq!(gene.id, 1);
    assert_eq!(gene.name, "test");
    assert_eq!(gene.value, Value::Int(42));
    assert_eq!(gene.weight, 0.5);
}

#[test]
fn test_mutation_application() -> std::io::Result<()> {
    let db = DNADatabase::builder()
        .data_dir(tempdir().unwrap().to_str().unwrap())
        .build()?;
}
```

```

    let id = db.create_genome(vec![], vec![])?;
    assert!(db.get_genome(id)?.is_some());

    Ok(())
}

```

15.2 Test Asynchrone (Future)

```

rust
// Proposition pour async/await
#[tokio::test]
async fn test_async_mutation() -> std::io::Result<> {
    let db = DNADatabase::new_async(config).await?;
    let id = db.create_genome(vec![], vec![]).await?;
    Ok(())
}

```

RÉFÉRENCE RAPIDE

Syntaxe	Description	Exemple
Gene::new(id, name, value, weight)	Crée un gène	Gene::new(1, "age".to_string(), Value::Int(30), 0.7)
Chromosome::new(id, name, genes, rate)	Crée un chromosome	Chromosome::new(1, "profile".to_string(), vec![1,2], 0.01)
Genome::new(id, chromosomes, genes)	Crée un genome	Genome::new(1, vec![c1], vec![g1,g2])
db.create_genome(chromosomes, genes)	Sauvegarde	db.create_genome(vec![c1], vec![g1])?
db.get_genome(id)	Lecture	db.get_genome(123)?
db.mutate(mutation)	Mutation	db.mutate(Mutation::SubstituteGene{...})?
db.top_genomes(n)	Top fitness	db.top_genomes(10)
db.search_similar(id, threshold)	Similarité	db.search_similar(123, 0.7)?
db.evolve(gens, prob)	Évolution	db.evolve(10, 0.2)?

Syntaxe	Description	Exemple
<code>db.gc()</code>	Nettoyage	<code>db.gc()?</code>

Cette documentation syntaxique couvre tous les aspects de DNA DB.

